

10 Shell Command Challenges

Combining grep, sed, and awk

A Collection of Data Processing Problems

September 20, 2025

Contents

1	Security Log: Failed Logins	2
2	Web Server Log: Bandwidth Calculation	4
3	CSV Data: Employee Report	5
3.1	Sample Input	5
4	Codebase Analysis: Deprecated Functions	6
5	System Config: Network Port Report	7
6	Application Log: Error Summary	8
7	Financial Data: Account Balance	9
8	Text Analysis: Shakespearean Word Count	10
9	DNS Log: External Queries	11
10	Apache Log: Referer Report	12

1 Security Log: Failed Logins

From your system's authentication log, find the **top 5 IP addresses** with the most "Failed password" attempts. For the final output, list the IP address and count, but **anonymize the last octet** (e.g., 198.51.100.22 becomes 198.51.100.XXX).

Sample Input (auth.log)

```
Sep 13 10:01:01 server sshd[123]: Failed password for invalid user
admin from 203.0.113.10 port 22
Sep 13 10:01:05 server sshd[124]: Failed password for root from
198.51.100.22 port 8080
Sep 13 10:01:09 server sshd[125]: Accepted publickey for user from
192.168.1.10
Sep 13 10:01:15 server sshd[126]: Failed password for root from
203.0.113.10 port 22
Sep 13 10:01:20 server sshd[127]: Failed password for user guest from
203.0.113.10 port 22
Sep 13 10:01:25 server sshd[128]: Failed password for root from
198.51.100.22 port 8080
Sep 13 10:01:30 server sshd[129]: Failed password for root from
203.0.113.55 port 22
```

Solution Command

```
grep 'Failed password' auth.log | awk '{print $(NF-3)}' | sort | uniq
-c | sort -nr | head -n 5 | sed -E 's/\.[0-9]{1,3}$/.XXX/' | awk
'{print $2 " (Count: " $1 ")"}'
```

Breakdown

1. `grep 'Failed password'`: Filters for lines indicating a failed login attempt.

```
Sep 13 10:01:01 server sshd[123]: Failed password for invalid
user admin from 203.0.113.10 port 22
Sep 13 10:01:05 server sshd[124]: Failed password for root from
198.51.100.22 port 8080
Sep 13 10:01:15 server sshd[126]: Failed password for root from
203.0.113.10 port 22
Sep 13 10:01:20 server sshd[127]: Failed password for user guest
from 203.0.113.10 port 22
Sep 13 10:01:25 server sshd[128]: Failed password for root from
198.51.100.22 port 8080
Sep 13 10:01:30 server sshd[129]: Failed password for root from
203.0.113.55 port 22
```

2. `awk '{print $(NF-2)}'`: Extracts the IP address. In this log format, it's the 4th field from the end (`NF-2`).

```
203.0.113.10
198.51.100.22
203.0.113.10
203.0.113.10
198.51.100.22
203.0.113.55
```

3. `sort | uniq -c | sort -nr | head -n 5`: A standard chain to count and find the top 5 IPs.

```
3 203.0.113.10
2 198.51.100.22
1 203.0.113.55
```

4. `sed -E 's/\.[0-9]{1,3}$/.XXX/': Anonymizes the IP. It finds a dot followed by 1 to 3 digits at the end of the line ($) and replaces it with .XXX.`
5. `awk '{print $2 " (Count: " $1 ")"}'`: Formats the final output. The count from `uniq -c` is \$1 and the IP is \$2.

Expected Output

```
203.0.113.XXX (Count: 3)
198.51.100.XXX (Count: 2)
203.0.113.XXX (Count: 1)
```

2 Web Server Log: Bandwidth Calculation

Calculate the **total bandwidth (in megabytes)** consumed by requests for `.zip` files made by the "Firefox" browser. Exclude any requests that resulted in a `404 Not Found` error.

Sample Input (`access.log`)

```
203.0.113.10 - - [...] "GET /d/a1.zip HTTP/1.1" 200 512000 "-" "
Mozilla/5.0 Firefox/125.0"
198.51.100.22 - - [...] "GET /d/a2.zip HTTP/1.1" 200 256000 "-" "
Mozilla/5.0 Chrome/127.0"
203.0.113.10 - - [...] "GET /d/none.zip HTTP/1.1" 404 209 "-" "
Mozilla/5.0 Firefox/125.0"
203.0.113.10 - - [...] "GET /d/a3.zip HTTP/1.1" 200 1024000 "-" "
Mozilla/5.0 Firefox/125.0"
```

Solution Command

```
grep '\.zip' access.log | grep 'Firefox' | grep -v ' 404 ' | awk '{
    sum+=$10} END {printf "%.2f MB\n", sum/(1024*1024)}'
```

Breakdown

1. `grep '\.zip'` and `grep 'Firefox'`: Filter for lines containing both patterns.
2. `grep -v ' 404 '`: Excludes lines containing the `404` status code.
3. `awk '{...}'`: For each matching line, adds the byte size (field 10) to a `sum`. After processing all lines, the `END` block calculates the total in MB and prints it.

Expected Output

```
1.46 MB
```

3 CSV Data: Employee Report

From an `employees.csv` file (`ID, Name, Dept, Salary, StartDate`), report all employees in the "Sales" department who started **before 2022**. The output should be `Name --- Salary: $X,XXX`, with a comma for thousands.

3.1 Sample Input

```
101,John Doe,Sales,85000,2021-06-15
102,Jane Smith,Engineering,110000,2020-03-10
103,Peter Jones,Sales,120000,2023-01-20
104,Mary Williams,Sales,95500,2019-11-01
```

Solution Command

```
grep 'Sales' employees.csv | awk -F, '{split($5, a, "-"); if (a[1] < 2022) print $2 "---" $4}' | sed -E ':a;s/\B[0-9]{3}\>/,/&/;ta' | sed -E 's/---/ --- Salary: $/g'
```

Breakdown

1. `grep 'Sales'`: Filters for lines containing "Sales".
2. `awk -F, '{...}'`: Sets the delimiter to a comma. It splits the date field (\$5) to get the year and, if the year is less than 2022, prints the name and salary separated by ---.
3. `sed -E ':a;s/\B[0-9]{3}\>/,/&/;ta'`: A classic `sed` loop to insert commas into numbers. It repeatedly finds a group of 3 digits at a word boundary and prepends a comma.
4. `sed -E 's/---/ --- Salary: $/g'`: Formats the final string.

Expected Output

```
John Doe --- Salary: $85,000
Mary Williams --- Salary: $95,500
```

4 Codebase Analysis: Deprecated Functions

Find all occurrences of a deprecated function `getLegacyData(arg1, arg2)` in a codebase. Generate a `sed` script that will replace these calls with `new_function(arg2)`.

Sample Input (various .js files)

```
// file1.js
const data = getLegacyData("config1", options);

// file2.js
let result = getLegacyData(user, settings);
```

Solution Command

```
grep -lr 'getLegacyData(' . | awk '{print "sed -i \047s/getLegacyData([^\,]*, \([^)]*\))/new_function(\1)/g\047 " $0}'
```

Breakdown

1. `grep -lr 'getLegacyData(' .`: Recursively (`-r`) searches the current directory (`.`) for the function string and lists (`-l`) only the filenames that contain it.
2. `awk '{...}'`: For each filename received, `awk` constructs a complete `sed` command.
 - `\047` is the octal code for a single quote, used to embed quotes within the printed string.
 - The regex captures the second argument and uses it as a backreference (`\1`) in the replacement.
 - `$0` refers to the entire input line (the filename).

Expected Output

```
sed -i 's/getLegacyData([^\,]*, \([^)]*\))/new_function(\1)/g' ./file1.js
sed -i 's/getLegacyData([^\,]*, \([^)]*\))/new_function(\1)/g' ./file2.js
```

5 System Config: Network Port Report

From the output of `netstat -tuln`, create a report of all listening services on `localhost` (127.0.0.1). Map common ports to service names.

Sample Input (from `netstat -tuln`)

```
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 127.0.0.1:6379 0.0.0.0:* LISTEN
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN
tcp 0 0 127.0.0.1:5432 0.0.0.0:* LISTEN
tcp6 0 0 :::80 :::* LISTEN
```

Solution Command

```
netstat -tuln | grep 'LISTEN' | sed -E 's/ +/ /g' | awk '{split($4, a
, ":"); port=a[length(a)]; service="Unknown"; if(port==22)service
="SSH"; if(port==80)service="HTTP"; if(port==6379)service="Redis";
if(port==5432)service="PostgreSQL"; print service " (Port: " port
)"}'
```

Breakdown

1. `grep 'LISTEN'`: Filters for listening services.
2. `sed -E 's/ +/ /g'`: Standardizes whitespace, collapsing multiple spaces into one.
3. `awk '{...}'`: This is the core logic.
 - It splits the Local Address field (\$4) by the colon to get the port number.
 - It uses a series of *if* statements to map the port to a known service name.
 - Finally, it prints the formatted string.

Expected Output

```
Redis (Port: 6379)
SSH (Port: 22)
PostgreSQL (Port: 5432)
HTTP (Port: 80)
```

6 Application Log: Error Summary

From a log with errors like `[ERROR] [Timestamp] Details...`, create a summary of unique error messages that occurred between 02:00 and 03:00 AM, stripping timestamps and UUIDs for grouping.

Sample Input (`app.log`)

```
[INFO] [2025-09-13T01:55:00] User logged in
[ERROR] [2025-09-13T02:10:15] Failed payment for user_id=a1b2c3d4,
    transaction_id=11223344-5566-7788-9900-aabbccddeeff
[ERROR] [2025-09-13T02:30:45] Failed payment for user_id=e4f5g6h7,
    transaction_id=fedcba98-7654-3210-fedc-ba9876543210
[ERROR] [2025-09-13T03:05:00] Database connection timeout
```

Solution Command

```
grep '\[ERROR\]' app.log | grep 'T02:' | sed -E 's/\[ERROR\]
\[^[^]]+\] //' | sed -E 's/[a-f0-9-]{8,}(\,)?//g' | sort | uniq -c
```

Breakdown

1. `grep '\[ERROR\]'`: Filters for error lines.
2. `grep 'T02:'`: Filters for lines with a timestamp in the 02:00 hour.
3. `sed -E 's/\[ERROR\] \[^[^]]+\] //'`: Removes the `[ERROR] [Timestamp]` prefix.
4. `sed -E 's/[a-f0-9-]{8,}(\,)?//g'`: Removes UUIDs and other long alphanumeric IDs to help group similar errors.
5. `sort | uniq -c`: Counts the occurrences of the cleaned, unique error messages.

Expected Output

```
2 Failed payment for user_id= transaction_id=
```


7 Financial Data: Account Balance

From a *transactions.txt* file with deposits and withdrawals, calculate the final account balance, ignoring comment lines.

Sample Input (*transactions.txt*)

```
# Initial balance: 0
DEPOSIT: +$1,250.50
WITHDRAWAL: -$75.00
DEPOSIT: +$300.00
# End of month
WITHDRAWAL: -$450.25
```

Solution Command

```
grep -v '^#' transactions.txt | sed -E 's/[A-Z:]+ //; s/[$+]/g; s/,//g' | awk '{balance+=$1} END {printf "Final Balance: $%.2f\n", balance}'
```

Breakdown

1. `grep -v '^#'`: Excludes all comment lines that start with #.
2. `sed -E 's/[A-Z:]+ //; s/[$+]/g; s/,//g'`: Performs multiple substitutions to clean the data: removes the text prefix, removes dollar signs and plus signs, and removes thousand-separator commas.
3. `awk '{balance+=$1} END {printf "Final Balance: $%.2f\n", balance}'`: For each cleaned line, adds the value to a *balance* variable. The *END* block prints the final formatted sum.

Expected Output

```
Final Balance: $1025.25
```

8 Text Analysis: Shakespearean Word Count

From a plain text file of Shakespeare's *Hamlet*, find the top 5 most frequently used **five-letter words**, case-insensitively, excluding the stop words "the", "and", "but", "for".

Sample Input (*hamlet.txt*)

```
To be, or not to be, that is the question:
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles
And by opposing end them. To -diety sleep,
No more; and by a sleep to say we end
The heart-ache and the thousand natural shocks
```

Solution Command

```
grep -o -E '\w+' hamlet.txt | awk 'length($0) == 5' | sed 'y/
ABCDEFGHIJKLMNOPQRSTUVWXYZ/abcdefghijklmnopqrstuvwxyz/' | grep -v
-w -E '^(the|and|but|for)$' | sort | uniq -c | sort -nr | head -n
5
```

Breakdown

1. `grep -o -E '\w+'`: Extracts all words, printing each on a new line.
2. `awk 'length($0) == 5'`: Filters for words that are exactly five letters long.
3. `sed 'y/.../.../'`: Converts all words to lowercase.
4. `grep -v -w -E '^(...)$'`: Excludes the specified stop words. `-w` ensures whole-word matching.
5. `sort | uniq -c | ...`: Counts, sorts, and selects the top 5 words.

Expected Output

```
3 sleep
2 more
1 say
1 them
1 take
```

9 DNS Log: External Queries

From a DNS query log, find all unique queries for `.com` domains not from your local network `192.168.1.0/24`. Remove any `www.` prefix.

Sample Input (`dns.log`)

```
13/Sep/2025:11:00:15 192.168.1.50 A? www.google.com.
13/Sep/2025:11:01:05 203.0.113.10 A? api.github.com.
13/Sep/2025:11:02:11 203.0.113.10 A? www.google.com.
13/Sep/2025:11:02:45 1.1.1.1 A? download.kernel.org.
13/Sep/2025:11:03:00 208.67.222.222 A? example.com.
```

Solution Command

```
grep '\.com' dns.log | sed '/ 192\.168\.1\./d' | awk '{print $8}' |
sed -E 's/^www\.//; s/\.$//;' | sort | uniq
```

Breakdown

1. `grep '\.com'`: Filters for lines with `.com` queries.
2. `sed '/ 192\.168\.1\./d'`: Deletes lines containing an IP from the local network range.
3. `awk '{print $8}'`: Extracts the domain name (field 8).

4. `sed -E 's/^www\.//; s/\.$//'`

: Removes the `www.` prefix and the trailing dot.

5. `sort | uniq`: Finds the unique domain names.

Expected Output

```
api.github.com
example.com
google.com
```

10 Apache Log: Referer Report

From an `access.log`, find the top 3 external domains referring traffic to your `/blog/` section. Exclude your own domain.

Sample Input (`access.log`)

```
... "GET /blog/post1 HTTP/1.1" 200 ... "https://www.google.com/search?q=stuff" ...
... "GET /blog/post2 HTTP/1.1" 200 ... "https://t.co/xyz" ...
... "GET /contact HTTP/1.1" 200 ... "https://www.google.com/" ...
... "GET /blog/post3 HTTP/1.1" 200 ... "https://news.ycombinator.com/" ...
... "GET /blog/post4 HTTP/1.1" 200 ... "https://www.yourdomain.com/internal" ...
... "GET /blog/post5 HTTP/1.1" 200 ... "-" ...
... "GET /blog/post6 HTTP/1.1" 200 ... "https://www.google.com/" ...
```

Solution Command

```
grep '"GET /blog/' access.log | awk -F'"' '{print $6}' | grep -v -E 'yourdomain.com|^-?$' | sed -E 's|https?://([^\s]+)/.*|\1|' | sort | uniq -c | sort -nr | head -n 3 | awk '{print NR". " $2 " - " $1}'
```

Breakdown

1. `grep '"GET /blog/`: Filters for GET requests to the blog section.
2. `awk -F'"' '{print $6}'`: Sets the delimiter to a double quote and prints the 6th field, which is the referer URL.
3. `grep -v -E '...'`: Excludes self-referrals (`yourdomain.com`) and empty (-) or invalid referers.
4. `sed -E 's|...|\1|'`: Extracts just the domain name from the full URL.
5. `sort | uniq -c | ...`: Counts, sorts, and gets the top 3 domains.
6. `awk '{print NR". " ...}'`: Formats the final output with a rank number (`NR`).

Expected Output

```
1. www.google.com - 2
2. news.ycombinator.com - 1
3. t.co - 1
```